FIAP - Faculdade de Informática e Administração Paulista

Pós-Graduação em Inteligência Artificial para Devs

ASSISTENTE VIRTUAL

MÉDICO COM IA GENERATIVA

Fine-Tuning de LLM, RAG com Protocolos Clínicos e Grafo Multi-Agente com LangGraph

TECH CHALLENGE - FASE 3

Grupo "Sala 14"

Adriana Martins de Souza - RM 368050
Diego Oliveira da Silva - RM 367964
Eduardo Nicola F. Zagari - RM 368021
Renan de Assis Torres - RM 368513

São Paulo

Março de 2026

Grupo "Sala 14"

ASSISTENTE VIRTUAL MÉDICO COM IA GENERATIVA

Fine-Tuning de LLM com MedQuAD, RAG com Protocolos Clínicos e Orquestração Multi-Agente com LangGraph

Relatório técnico apresentado como parte dos requisitos para aprovação no Tech Challenge da Fase 3 do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP.

São Paulo

Março de 2026

Disponibilização do Código e Materiais

Todo o código desenvolvido neste projeto, bem como esta documentação e o link para o vídeo demonstrativo estão disponíveis publicamente em:

GitHub: https://github.com/Zagari/virtual-medical-assistant

Conteúdo Disponível:

- Pipeline completo de reprodução com 7 scripts numerados (`scripts/01` a `scripts/07`)
- Código-fonte do fine-tuning (QLoRA) e notebook para Google Colab
- 14 Linhas de Cuidado + 5 protocolos de emergência (IAM, AVC, Sepse, Pneumonia, ITU) para RAG
- Dados sintéticos de 80 pacientes com exames, prontuários e receitas
- Grafo multi-agente LangGraph com 6 agentes especializados
- Este relatório técnico em PDF

Vídeo demonstrativo: https://youtube.com/TODO

Licença: MIT License (código aberto para fins educacionais e de pesquisa)

Resumo

Este documento apresenta a documentação técnica completa do Assistente Virtual Médico desenvolvido como parte do Tech Challenge da Fase 3 do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP. O sistema integra cinco pilares de Inteligência Artificial (IA) generativa: (1) **fine-tuning** do modelo Mistral 7B Instruct v0.3 com o dataset MedQuAD traduzido para português, utilizando Quantized Low-Rank Adaptation (QLoRA) 4-bit no Google Colab; (2) **RAG** (*Retrieval-Augmented Generation*) com 14 Linhas de Cuidado reais e 5 protocolos de emergência sintéticos indexados no ChromaDB; (3) **grafo multi-agente** em LangGraph com 6 agentes especializados (triagem, protocolo, paciente, raciocínio, guardrails e explicabilidade); (4) **segurança** com guardrails que impedem prescrições diretas e exigem validação humana; e (5) **interface** Gradio para demonstração interativa.

O projeto foi concebido para ser integralmente reprodutível: um pipeline de 7 scripts numerados permite que se reconstrua todo o sistema do zero, desde a geração de dados sintéticos até a execução do assistente completo. Os dados de pacientes são inteiramente fictícios, gerados com coerência clínica para simular cenários reais de atendimento hospitalar.

Palavras-chave: Fine-Tuning; QLoRA; Mistral 7B; LangChain; LangGraph; RAG; ChromaDB; Multi-Agente; Guardrails; Assistente Médico.

Abstract

This document presents the complete technical documentation of the Virtual Medical Assistant developed as part of the Phase 3 Tech Challenge of the Postgraduate course in Artificial Intelligence for Developers at FIAP. The system integrates five pillars of generative AI: (1) **fine-tuning** of Mistral 7B Instruct v0.3 with the MedQuAD dataset translated to Portuguese, using QLoRA 4-bit on Google Colab; (2) **RAG** (Retrieval-Augmented Generation) with 14 real clinical care pathways and 5 synthetic emergency protocols indexed in ChromaDB; (3) **multi-agent graph** in LangGraph with 6 specialized agents (triage, protocol, patient data, reasoning, guardrails, and explainability); (4) **safety** guardrails preventing direct prescriptions and requiring human validation; and (5) **Gradio interface** for interactive demonstration.

The project was designed to be fully reproducible: a pipeline of 7 numbered scripts allows the entire system to be rebuilt from scratch, from synthetic data generation to running the complete assistant. Patient data is entirely fictitious, generated with clinical coherence to simulate real hospital care scenarios.

Keywords: Fine-Tuning; QLoRA; Mistral 7B; LangChain; LangGraph; RAG; ChromaDB; Multi-Agent; Guardrails; Medical Assistant.

Sumário

Disponibilização do Código e Materiais i

Resumo ii

Abstract iii

Lista de Abreviaturas e Siglas vii

1 Introdução 1

1.1 Contexto e Motivação 1

1.2 Objetivos do Projeto 1

1.3 Arquitetura Geral do Sistema 2

1.4 Guia de Reprodução Passo a Passo 3

1.4.1 Pipeline de Scripts 3

1.4.2 Fluxo de Execução 4

1.4.3 Como Reproduzir 4

1.5 Estrutura de Arquivos do Projeto 5

1.6 Organização do Documento 6

2 Dados e Preparação 7

2.1 Linhas de Cuidado (Protocolos Clínicos) 7

2.1.1 Protocolos Disponíveis 7

2.1.2 Extração e Limpeza dos Protocolos 8

2.1.3 Protocolos Complementares de Emergência 9

2.2 Dados Sintéticos de Pacientes 9

2.2.1 Geração de Pacientes 10

2.2.2 Coerência Clínica 10

2.2.3 Dados Clínicos Gerados 11

2.3 Banco de Dados PostgreSQL 11

2.3.1 Schema do Banco 11

2.3.2 Características Técnicas 13

2.3.3 Infraestrutura Docker 13

2.4 Dataset MedQuAD . . . 13
2.4.1 Escolha da Versão . . . 14
2.4.2 Características do Dataset . . . 14
2.4.3 Pipeline de Preparação . . . 14

3 Fine-Tuning do Modelo 16
3.1 Modelo Base: Mistral 7B Instruct . . . 16
3.1.1 Justificativa da Escolha . . . 16
3.1.2 Características do Mistral 7B . . . 16
3.2 Dataset: MedQuAD . . . 17
3.2.1 Escolha do Dataset . . . 17
3.2.2 Fontes do MedQuAD . . . 17
3.2.3 Tradução para Português . . . 18
3.2.4 Formato Alpaca . . . 19
3.3 Técnica: QLoRA . . . 20
3.3.1 Fundamentos do LoRA . . . 21
3.3.2 Quantização 4-bit . . . 21
3.3.3 Configuração do LoRA . . . 22
3.4 Processo de Treino . . . 22
3.4.1 Ambiente de Execução . . . 22
3.4.2 Hiperparâmetros de Treinamento . . . 23
3.4.3 Sistema de Checkpoints para Colab Free . . . 24
3.4.4 Estimativa de Tempo . . . 26
3.5 Persistência do Modelo . . . 26
3.5.1 Salvando os Adapters LoRA . . . 26
3.5.2 Carregando o Modelo para Uso . . . 26
3.6 Avaliação do Fine-Tuning . . . 27
3.6.1 Metodologia . . . 27
3.6.2 Conjunto de Perguntas de Avaliação . . . 27
3.6.3 Métricas de Avaliação . . . 28
3.6.4 Scripts de Avaliação . . . 28
3.6.5 Resultados Esperados . . . 28
3.7 Resumo do Capítulo . . . 29

4 RAG: Retrieval-Augmented Generation 30
4.1 Motivação e Arquitetura . . . 30
4.1.1 Componentes do Sistema . . . 30
4.2 Modelo de Embeddings . . . 31
4.2.1 Análise de Alternativas . . . 31
4.2.2 Modelo Selecionado: E5 Multilingual . . . 31

4.3 Estratégia de Chunking 32
4.3.1 Abordagens Consideradas 32
4.3.2 Implementação: Chunking Semântico 32
4.3.3 Exemplo de Chunking 33
4.4 Indexação no ChromaDB 34
4.4.1 Processo de Indexação 34
4.4.2 Estrutura dos Metadados 35
4.4.3 Estatísticas da Indexação 35
4.5 Recuperação de Contexto 35
4.5.1 Processo de Busca 36
4.5.2 Exemplo de Recuperação 36
4.5.3 Formatação do Contexto 37
4.6 Integração com o LLM 37
4.7 Validação do Sistema RAG 38
4.7.1 Métricas de Qualidade 38
4.7.2 Resultados dos Testes 39
4.8 Considerações de Desempenho 39
4.8.1 Tempo de Indexação 39
4.8.2 Tempo de Busca 39

Lista de Abreviaturas e Siglas

CID Classificação Internacional de Doenças. 13

IA Inteligência Artificial. ii, 1

LLM Large Language Model. 1, 19

LoRA Low-Rank Adaptation. 21

QLoRA Quantized Low-Rank Adaptation. ii, 1

RAG Retrieval-Augmented Generation. 2, 30

1

Introdução

1.1 Contexto e Motivação

A Inteligência Artificial (IA) generativa tem transformado diversas áreas do conhecimento, e o setor de saúde desponta como um dos campos com maior potencial de impacto. A capacidade dos Large Language Model (LLM) de processar, interpretar e gerar texto em linguagem natural abre oportunidades significativas para auxiliar profissionais de saúde em tarefas como consulta a protocolos clínicos, suporte à tomada de decisão e acesso rápido a informações de pacientes.

No entanto, modelos generalistas de linguagem — apesar de seu amplo conhecimento — apresentam limitações importantes no domínio médico: podem gerar informações imprecisas, desatualizadas ou descontextualizadas. Essas limitações tornam necessário o uso de técnicas especializadas como **fine-tuning** (ajuste fino) para aprimorar o conhecimento médico do modelo, **RAG** (*Retrieval-Augmented Generation*) para fundamentar respostas em protocolos clínicos reais, e **guardrails** de segurança para garantir que o sistema nunca substitua o julgamento clínico humano.

Este projeto foi desenvolvido no contexto do **Tech Challenge da Fase 3** do curso de Pós-Graduação em Inteligência Artificial para Devs da FIAP, cujo tema central é **IA Generativa**. O desafio propõe a construção de um assistente virtual inteligente que integre técnicas de fine-tuning de LLMs, orquestração multi-agente com LangGraph e segurança por design.

1.2 Objetivos do Projeto

O objetivo principal deste trabalho é desenvolver um **Assistente Virtual Médico** capaz de auxiliar profissionais de saúde no acesso a informações clínicas contextualizadas. Para isso, o sistema integra cinco pilares fundamentais:

1. **Fine-tuning** do modelo Mistral 7B Instruct v0.3 com o dataset MedQuAD traduzido para português, utilizando Quantized Low-Rank Adaptation (QLoRA) 4-bit no Google Colab;

2. **Retrieval-Augmented Generation (RAG)** com 14 Linhas de Cuidado reais e 5 protocolos de emergência sintéticos (IAM, AVC, Sepse, Pneumonia, ITU) indexados no ChromaDB;
3. **Grafo multi-agente** implementado em LangGraph com 6 agentes especializados;
4. **Segurança** com guardrails que impedem prescrições diretas e exigem validação humana;
5. **Interface** Gradio para demonstração interativa do assistente.

Aviso de Segurança

Este sistema foi projetado exclusivamente como ferramenta de **suporte à decisão clínica**, nunca como substituto do profissional de saúde. Todas as respostas incluem avisos de validação humana obrigatória e o sistema é impedido, por guardrails, de prescrever medicamentos diretamente.

1.3 Arquitetura Geral do Sistema

A Figura 1.1 apresenta a visão de alto nível da arquitetura, organizada em cinco camadas que correspondem aos cinco pilares do projeto.

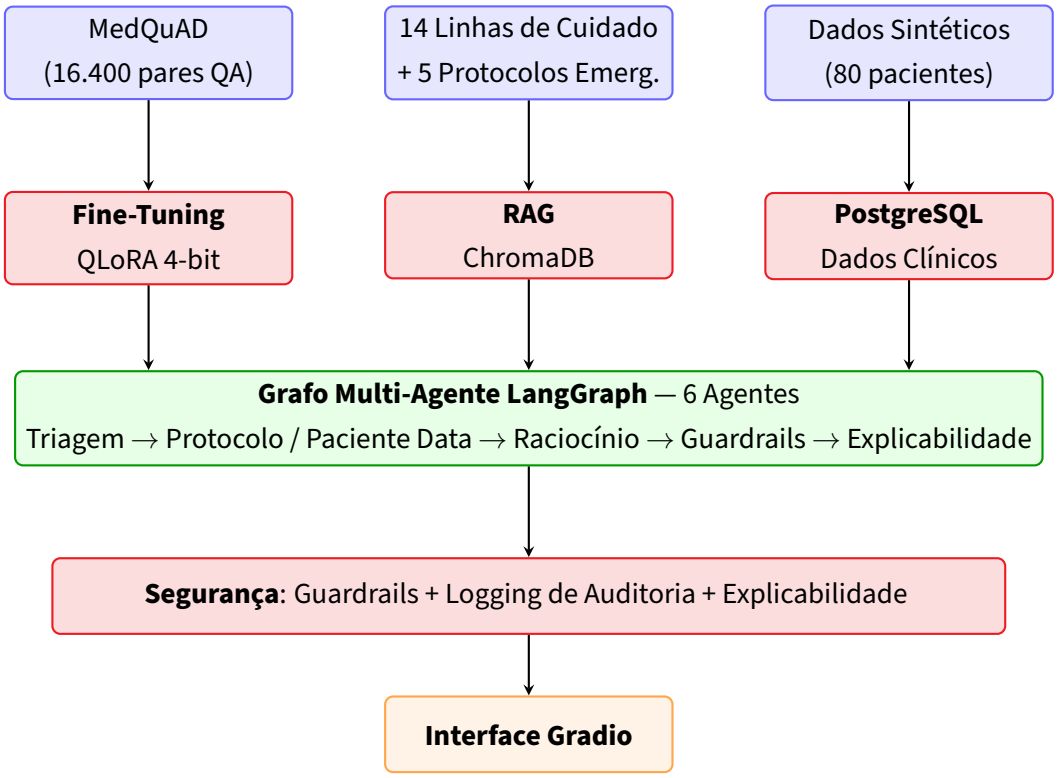

Figura 1.1: Arquitetura geral do Assistente Virtual Médico.

1.4 Guia de Reprodução Passo a Passo

O projeto foi concebido para ser **integralmente reprodutível**. Um pipeline de 7 scripts numerados, localizado no diretório `scripts/`, permite que se reconstrua todo o sistema do zero, desde a geração de dados sintéticos até a execução do assistente completo.

1.4.1 Pipeline de Scripts

A Tabela 1.1 descreve cada script, suas pré-condições e saídas.

Tabela 1.1: Pipeline de reprodução: scripts numerados.

#	Script	O que faz	Saída
01	`gerar_dados_sinteticos`	Gera pacientes, exames, prontuários, receitas e 5 protocolos de emergência	`synthetic/`
02	`setup_postgres`	Cria container Docker PostgreSQL e popula com dados sintéticos	Banco operacional
03	`preparar_dataset`	Baixa MedQuAD, traduz para PT-BR e formata em Alpaca	`training_data.jsonl`
–	*Pausa: Colab*	*Fine-tuning QLoRA no notebook* `03_fine_tuning.ipynb`	`models/`
04	`indexar_protocolos`	Chunking dos protocolos Markdown e indexação no ChromaDB	`chroma_db/`
05	`avaliar_baseline`	Executa 50 perguntas médicas no Mistral 7B base	`baseline.json`
06	`avaliar_finetuned`	Mesmas perguntas no modelo fine-tuned + comparação	Relatório comparativo
07	`iniciar_app`	Inicia o assistente completo com interface Gradio	Servidor web local

1.4.2 Fluxo de Execução

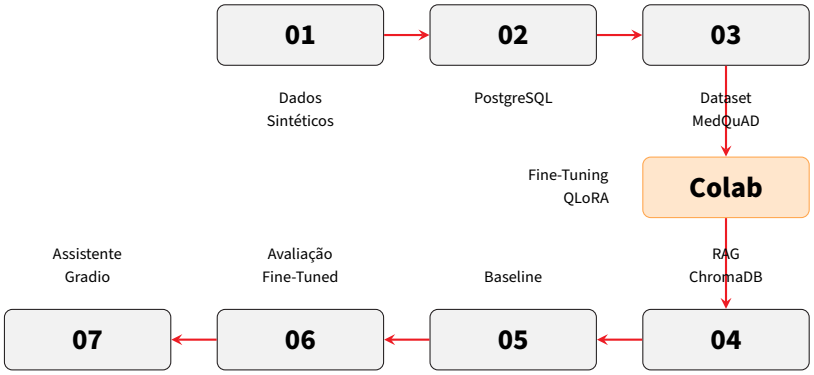

Figura 1.2: Fluxo de execução do pipeline de reprodução.

1.4.3 Como Reproduzir

```
# 0. Pre-requisitos: Python 3.11+, Docker Desktop
git clone
   ↪ https://github.com/Zagari/virtual-medical-assistant.git
cd virtual-medical-assistant
pip install -r requirements.txt

# 1. Gerar dados sinteticos (80 pacientes ficticios)
python scripts/01_gerar_dados_sinteticos.py

# 2. Subir PostgreSQL e popular banco
python scripts/02_setup_postgres.py

# 3. Preparar dataset MedQuAD para fine-tuning
python scripts/03_preparar_dataset.py

# --- PAUSA: Fine-tuning no Google Colab ---
# Abrir notebooks/03_fine_tuning.ipynb no Colab
# Upload de data/processed/training_data.jsonl
# Treinar modelo -> Download para models/

# 4. Indexar protocolos clinicos no ChromaDB
python scripts/04_indexar_protocolos.py

# 5-6. Avaliar modelo (baseline vs fine-tuned)
python scripts/05_avaliar_baseline.py
python scripts/06_avaliar_finetuned.py
```

```
# 7. Iniciar o assistente
python scripts/07_iniciar_app.py
```

Listing 1.1: Comandos para reprodução completa do projeto.

Princípios do Pipeline

Cada script é **autocontido** e pode ser executado independentemente. Todos imprimem progresso durante a execução (registros gerados, tempo decorrido) e **validam pré-condições** antes de iniciar (por exemplo, o script 02 verifica se os arquivos de dados sintéticos existem antes de tentar popular o banco).

1.5 Estrutura de Arquivos do Projeto

```
virtual-medical-assistant/
|-- README.md
|-- requirements.txt
|-- .env.example
|-- scripts/                    # Pipeline numerado (01 a 07)
|-- data/
|   |-- raw/                    # MedQuAD original
|   |-- processed/              # Dataset formatado
  ↪ (training_data.jsonl)
|   |-- linhas_de_cuidado/      # 14 PDFs + extracted/
  ↪ (Markdown)
|   +-- synthetic/              # Pacientes, exames,
  ↪ prontuarios, receitas
|-- notebooks/
|   +-- 03_fine_tuning.ipynb    # Notebook para Google Colab
|-- src/
|   |-- data/                   # Geracao e preprocessamento
|   |-- fine_tuning/            # Configuracao e treino
|   |-- assistant/              # LLM wrapper, RAG, prompts
|   |-- flows/                  # LangGraph (state, graph,
  ↪ agents/)
|   |-- security/               # Guardrails, logging, auditoria
|   +-- database/               # SQLAlchemy models e queries
|-- app/                        # Interface Gradio
|-- docs/                       # Relatorio LaTeX
+-- tests/                      # Testes automatizados
```

Listing 1.2: Estrutura de diretórios do projeto.

1.6 Organização do Documento

O restante deste relatório está organizado da seguinte forma:

- **Capítulo 2** apresenta os dados utilizados: as 14 Linhas de Cuidado, a geração de dados sintéticos, o banco de dados PostgreSQL e o dataset MedQuAD;
- **Capítulo 3** detalha o processo de fine-tuning do Mistral 7B com QLoRA;
- **Capítulo 4** descreve a implementação do RAG com ChromaDB;
- **Capítulo 5** apresenta o grafo multi-agente com LangGraph;
- **Capítulo 6** aborda a segurança, logging e interface Gradio;
- **Capítulo 7** traz a avaliação e os resultados obtidos;
- **Capítulo 8** conclui com reflexões, limitações e trabalhos futuros.

2

Dados e Preparação

Este capítulo descreve as quatro fontes de dados do projeto: os protocolos clínicos reais (Linhas de Cuidado), os dados sintéticos de pacientes, o banco de dados PostgreSQL e o dataset MedQuAD para fine-tuning.

2.1 Linhas de Cuidado (Protocolos Clínicos)

A base de conhecimento clínico do sistema é composta por **14 Linhas de Cuidado** — protocolos clínicos profissionais em português brasileiro, cobrindo condições que vão de hipertensão e diabetes até demência e polifarmácia. Estes protocolos são documentos reais de prática clínica, com estrutura padronizada que inclui seções como Considerações Iniciais, Diagnóstico, Monitoramento, Encaminhamento e Indicadores.

2.1.1 Protocolos Disponíveis

A Tabela 2.1 lista os 14 protocolos utilizados como base de conhecimento para o sistema RAG.

Tabela 2.1: 14 Linhas de Cuidado indexadas no sistema.

#	Pathway	Domínio	Exemplos de Conteúdo
1	Hipertensão	Cardiovascular	Classificação PA, MAPA, anti-hipertensivos
2	Diabetes	Endócrino	DM1/DM2, HbA1C, insulinização
3	Asma	Respiratório	Classificação gravidade, step therapy
4	Depressão	Psiquiátrico	PHQ-9, ISRS, psicoterapia
5	Ansiedade	Psiquiátrico	GAD-7, benzodiazepínicos, TCC
6	DPOC	Respiratório	GOLD, espirometria, oxigenoterapia
7	Demência	Neurológico	Mini-Mental, anticolinesterásicos
8	Dispepsia	Gastroenterologia	H. pylori, IBP, endoscopia
9	Hipotireoidismo	Endócrino	TSH, levotiroxina, seguimento
10	Insuf. Cardíaca	Cardiovascular	NYHA, IECA, betabloqueadores
11	Obesidade	Metabólico	IMC, dietoterapia, cirurgia bariátrica
12	Pac. Cirúrgico	Cirúrgico	Pré-operatório, risco cirúrgico
13	Polifarmácia	Geriatria	Reconciliação medicamentosa, Beers
14	Tabagismo	Preventivo	Fagerström, TRN, bupropiona

2.1.2 Extração e Limpeza dos Protocolos

Os 14 protocolos foram disponibilizados originalmente em formato PDF. Para viabilizar a indexação no sistema RAG, foi realizado um processo de extração e limpeza:

1. **Extração de texto:** Uso da biblioteca PyMuPDF (fitz) para extrair o conteúdo textual de cada página dos PDFs;
2. **Identificação de seções:** Detecção automática de seções principais por meio de heurística baseada em texto em caixa alta (*ALL CAPS*), seguindo o padrão estrutural dos documentos;
3. **Limpeza:** Remoção de referências ao sistema de origem e caracteres indesejados;
4. **Saída em Markdown:** Geração de arquivos `.md` com hierarquia de seções preservada (## para seções, ### para subseções), facilitando o chunking posterior.

O resultado são 14 arquivos Markdown limpos, totalizando aproximadamente 448 KB de conteúdo clínico estruturado, armazenados em `data/linhas_de_cuidado/extracted/`. Estes arquivos são versionados no repositório e servem como entrada direta para o pipeline de RAG (Capítulo 4).

Nota sobre a Extração

O script de extração de PDFs é um utilitário auxiliar executado uma única vez fora do pipeline principal. Os arquivos Markdown resultantes já estão incluídos no repositório, portanto **não é necessário reexecutar** a extração — basta utilizar os arquivos `.md` já disponíveis.

2.1.3 Protocolos Complementares de Emergência

Para cobrir cenários de **emergência** não presentes nas 14 Linhas de Cuidado (que focam em condições crônicas), o sistema inclui 5 protocolos sintéticos adicionais. Estes protocolos foram elaborados com base em diretrizes reconhecidas (AHA, Surviving Sepsis Campaign, SBPT, IDSA) e seguem a mesma estrutura dos protocolos extraídos.

Tabela 2.2: Protocolos de emergência sintéticos complementares.

#	Protocolo	CID	Conteúdo Principal
1	IAM (Infarto Agudo do Miocárdio)	I21	MONABCH, reperfusão, tempo porta-balão
2	AVC (Acidente Vascular Cerebral)	I64	NIHSS, trombólise rtPA, janela terapêutica
3	Sepse e Choque Séptico	A41	Hour-1 Bundle, qSOFA, ressuscitação
4	Pneumonia (PAC)	J18	CURB-65, antibioticoterapia empírica
5	ITU (Infecção do Trato Urinário)	N39	Cistite vs pielonefrite, bacteriúria

Cada protocolo sintético inclui as seções: Considerações Iniciais, Diagnóstico, Manejo Inicial, Monitoramento e Indicadores de Qualidade. Os arquivos são gerados automaticamente pelo script `01_gerar_dados_sinteticos` e salvos em formato Markdown no diretório `data/synthetic/protocolos_complementares/`, prontos para indexação no ChromaDB junto com as Linhas de Cuidado.

Protocolos Sintéticos

Os protocolos de emergência são **sintéticos**, criados para fins educacionais e de demonstração. Em ambiente de produção, devem ser substituídos por protocolos institucionais validados pela equipe médica.

2.2 Dados Sintéticos de Pacientes

Para simular um ambiente hospitalar realista, o sistema utiliza dados de pacientes inteiramente fictícios, gerados com coerência clínica. A geração é realizada pelo primeiro script

do pipeline (`01_gerar_dados_sinteticos`), que invoca o módulo `synthetic_generator` localizado em `src/data/`.

2.2.1 Geração de Pacientes

Foram gerados **80 pacientes fictícios** utilizando a biblioteca Faker (localização `pt_BR`), com sementes fixas (`seed=42`) para garantir a reprodutibilidade. Os dados pessoais incluem:

Tabela 2.3: Campos do cadastro de pacientes sintéticos.

Campo	Exemplo	Geração
nome	"Maria Santos de Oliveira"	`Faker.name()`
cpf	"123.456.789-00"	`Faker.cpf()`
data_nascimento	1958-03-15	`Faker.date_of_birth()`
sexo	"F"	Distribuição 55%F / 45%M
telefone	"(11) 98765-4321"	`Faker.phone_number()`
endereço	"Rua das Flores, 123"	`Faker.address()`
comorbidades	["hipertensão", "DM2"]	Distribuição probabilística
alergias	["dipirona"]	Lista aleatória
medicamentos_uso	["losartana 50mg"]	Coerente com comorbidades

2.2.2 Coerência Clínica

Um diferencial importante da geração de dados é a **coerência clínica** entre comorbidades, medicamentos, exames e consultas. O gerador implementa uma base de conhecimento médico com 9 condições alinhadas às Linhas de Cuidado:

Tabela 2.4: Distribuição de condições nos dados sintéticos.

Condição	Prevalência	Exemplos de Medicamentos Associados
Hipertensão	45%	Losartana 50mg, enalapril 10mg, anlodipino 5mg
Diabetes tipo 2	30%	Metformina 850mg, glibenclamida 5mg, insulina NPH
Obesidade	25%	Sibutramina 10mg, orlistate 120mg
Depressão	20%	Sertralina 50mg, fluoxetina 20mg, escitalopram 10mg
Ansiedade	15%	Escitalopram 10mg, alprazolam 0,5mg
Asma	12%	Salbutamol spray, budesonida 200mcg
Hipotireoidismo	10%	Levotiroxina 50mcg, levotiroxina 75mcg
DPOC	8%	Tiotrópio 18mcg, formoterol + budesonida
Insuf. Cardíaca	5%	Enalapril 10mg, furosemida 40mg, carvedilol 12,5mg

Muitos pacientes possuem **múltiplas comorbidades** (por exemplo, hipertensão + diabetes + obesidade), o que é clinicamente realista e cria cenários ricos para o assistente consultar múltiplos protocolos simultaneamente.

2.2.3 Dados Clínicos Gerados

Para cada paciente, o gerador cria dados clínicos coerentes com suas comorbidades:

Tabela 2.5: Volume de dados clínicos gerados.

Tipo	Total Gerado	Arquivo de Saída
Pacientes	80	`pacientes.json`
Exames	199	`exames.json`
Prontuários	297	`consultas.json`
Receitas	163	`receitas.json`

Exemplos de coerência clínica implementada:

- Paciente **diabético** → exames de HbA1C (valores entre 5,5–12%), glicemia de jejum (80–250 mg/dL), creatinina;
- Paciente **hipertenso** → MAPA 24h com valores sistólico/diastólico realistas, ecocardiograma;
- Paciente **asmático** → espirometria com VEF1/CVF e classificação de gravidade;
- Paciente com **depressão** → avaliação PHQ-9 com escore de gravidade e notas clínicas compatíveis.

2.3 Banco de Dados PostgreSQL

Os dados sintéticos são armazenados em um banco PostgreSQL 16, executado em container Docker. O setup é automatizado pelo `scripts/02_setup_postgres.py`.

2.3.1 Schema do Banco

O banco possui 4 tabelas, modeladas com SQLAlchemy ORM:

```sql
-- Pacientes
CREATE TABLE pacientes (
    id UUID PRIMARY KEY,
    nome VARCHAR(200) NOT NULL,
    cpf VARCHAR(14) UNIQUE NOT NULL,
```

```sql
    data_nascimento DATE NOT NULL,
    sexo VARCHAR(1) NOT NULL,
    telefone VARCHAR(20),
    endereco TEXT,
    comorbidades TEXT[],
    alergias TEXT[],
    medicamentos_uso TEXT[]
);

-- Exames clinicos
CREATE TABLE exames (
    id UUID PRIMARY KEY,
    paciente_id UUID REFERENCES pacientes(id),
    tipo VARCHAR(100) NOT NULL,
    data DATE NOT NULL,
    resultados JSONB,        -- Resultados estruturados
    interpretacao TEXT,
    status VARCHAR(20)       -- pendente, realizado, cancelado
);

-- Prontuarios (consultas medicas)
CREATE TABLE prontuarios (
    id UUID PRIMARY KEY,
    paciente_id UUID REFERENCES pacientes(id),
    data DATE NOT NULL,
    diagnostico_cid VARCHAR(10),  -- Codigo CID-10
    diagnostico_texto TEXT,
    observacoes TEXT
);

-- Receitas medicas
CREATE TABLE receitas (
    id UUID PRIMARY KEY,
    paciente_id UUID REFERENCES pacientes(id),
    data DATE NOT NULL,
    condicao TEXT,
    medicamentos TEXT[],
    validade_dias INTEGER,
    observacoes TEXT
);
```

Listing 2.1: Schema do banco de dados.

2.3.2 Características Técnicas

- **UUIDs como chaves primárias:** Evita colisões e permite geração distribuída;
- **JSONB para resultados de exames:** Permite armazenar resultados com estrutura variável (diferentes tipos de exames geram diferentes campos), mantendo a capacidade de consulta;
- **ARRAY para listas:** Comorbidades, alergias e medicamentos são armazenados como arrays nativos do PostgreSQL, facilitando consultas com operador `ANY`;
- **CID-10:** Diagnósticos são codificados no padrão Classificação Internacional de Doenças (CID), permitindo consultas padronizadas;
- **Consultas parametrizadas:** O módulo `src/database/queries.py` utiliza `SQLAlchemy text()` com parâmetros nomeados para prevenir injeção SQL.

2.3.3 Infraestrutura Docker

O script `02_setup_postgres.py` automatiza todo o processo:

1. Verifica se o Docker Desktop está em execução;
2. Cria ou inicia o container `medical_assistant_db` (PostgreSQL 16 Alpine);
3. Cria as 4 tabelas via `SQLAlchemy Base.metadata.create_all()`;
4. Carrega os 4 arquivos JSON gerados pelo script 01;
5. Popula o banco de dados com os dados sintéticos;
6. Executa validação final (contagem de registros por tabela).

2.4 Dataset MedQuAD

O **MedQuAD** (*Medical Question Answering Dataset*) é o dataset utilizado para o fine-tuning do modelo Mistral 7B. Criado pelo National Institutes of Health (NIH) dos Estados Unidos, o MedQuAD reúne pares de perguntas e respostas médicas extraídas de fontes confiáveis do governo americano.

2.4.1 Escolha da Versão

Existem múltiplas versões do MedQuAD disponíveis no HuggingFace. Após análise comparativa, optou-se pela versão `lavita/MedQuAD`, a mais completa:

Tabela 2.6: Comparação de versões do MedQuAD disponíveis no HuggingFace.

Dataset	Pares QA	Metadados	Observações
`lavita/MedQuAD`	**47.400**	UMLS, fontes, sinônimos	Versão mais completa
`keivalya/MedQuad-...`	16.400	Apenas tipo de questão	Versão simplificada
`Tonic/medquad`	15.549	Nenhum	Pré-formatado Alpaca

2.4.2 Características do Dataset

Tabela 2.7: Resumo do dataset MedQuAD (versão lavita).

Característica	Descrição
Tamanho	**47.400** pares pergunta-resposta
Idioma original	Inglês
Fontes	8 institutos NIH: NCI, GHR, MedlinePlus, NIDDK, NINDS, NIHSeniorHealth, NHLBI, CDC
Metadados	Códigos UMLS, tipos semânticos, sinônimos, URLs das fontes
Domínios	Doenças, tratamentos, medicamentos, prevenção, diagnósticos
Qualidade	Alta — respostas baseadas em fontes governamentais revisadas
Licença	Creative Commons BY 4.0
Disponibilidade	HuggingFace: `lavita/MedQuAD`

2.4.3 Pipeline de Preparação

O `scripts/03_preparar_dataset.py` executa o seguinte pipeline:

1. **Download:** Baixa o dataset `lavita/MedQuAD` do HuggingFace via biblioteca `datasets`;
2. **Tradução:** Traduz perguntas e respostas para português brasileiro utilizando a biblioteca `deep-translator` (Google Translate gratuito);
3. **Checkpoints:** Salva o progresso a cada 1.000 registros traduzidos, permitindo retomar em caso de interrupção;
4. **Formatação Alpaca:** Converte para o formato JSONL esperado pelo fine-tuning.

O formato Alpaca utilizado segue a estrutura padrão para instruction tuning:

```
{
    "instruction": "Responda a seguinte pergunta medica de forma
                    clara e detalhada.",
    "input": "Quais sao os sintomas do diabetes tipo 2?",
    "output": "Os sintomas comuns do diabetes tipo 2 incluem..."
}
```

Listing 2.2: Formato Alpaca para fine-tuning.

O arquivo de saída (`data/processed/training_data.jsonl`) contém aproximadamente 47.400 registros traduzidos, totalizando cerca de 100–200 MB.

Por que a versão lavita/MedQuAD?

A versão `lavita/MedQuAD` foi escolhida por oferecer: (1) **maior volume** — 47.400 pares, quase 3x mais que alternativas; (2) **metadados ricos** — códigos UMLS, tipos semânticos e sinônimos úteis para RAG; (3) **fontes documentadas** — 8 institutos do NIH com cobertura ampla de especialidades médicas; (4) **licença clara** — CC BY 4.0, permitindo uso acadêmico e comercial.

3

Fine-Tuning do Modelo

Este capítulo descreve o processo de fine-tuning do modelo de linguagem para o domínio médico. O objetivo é adaptar um modelo generalista (Mistral 7B) para responder perguntas médicas em português com maior precisão e relevância clínica.

3.1 Modelo Base: Mistral 7B Instruct

3.1.1 Justificativa da Escolha

Para o fine-tuning, escolhemos o **Mistral 7B Instruct v0.3** como modelo base. Esta decisão foi tomada após análise comparativa de alternativas, considerando os seguintes critérios:

Tabela 3.1: Comparação de modelos candidatos para fine-tuning

Modelo	Parâmetros	Contexto	Licença	VRAM (4-bit)
Mistral 7B Instruct	7.3B	32K tokens	Apache 2.0	5GB
LLaMA 2 7B	7B	4K tokens	Meta License	5GB
BioMistral 7B	7.3B	32K tokens	Apache 2.0	5GB
Falcon 7B	7B	2K tokens	Apache 2.0	5GB

Por que não usar BioMistral?

O BioMistral já possui conhecimento médico pré-treinado, o que dificultaria demonstrar o **delta** (melhoria) obtido com o fine-tuning. Ao usar o Mistral 7B generalista, conseguimos evidenciar claramente a diferença entre as respostas antes e depois do treinamento especializado.

3.1.2 Características do Mistral 7B

O Mistral 7B apresenta características técnicas que o tornam ideal para este projeto:

- **Arquitetura**: Transformer decoder-only com 32 camadas
- **Sliding Window Attention**: Janela de 4096 tokens com contexto efetivo de 32K
- **Grouped-Query Attention (GQA)**: Reduz uso de memória mantendo qualidade
- **Instruct-tuned**: Versão já otimizada para seguir instruções
- **Multilíngue**: Bom desempenho em português, mesmo sem treinamento específico

3.2 Dataset: MedQuAD

3.2.1 Escolha do Dataset

Para o fine-tuning, utilizamos o **MedQuAD** (Medical Question Answering Dataset), um dataset de perguntas e respostas médicas criado pelo National Institutes of Health (NIH) dos Estados Unidos.

Tabela 3.2: Comparação de versões do MedQuAD disponíveis

Dataset (HuggingFace)	Pares QA	Metadados	Escolhido
`lavita/MedQuAD`	47.400	UMLS, fontes, sinônimos	✓
`keivalya/MedQuad-MedicalQnADataset`	16.400	Apenas tipo de questão	
`Tonic/medquad`	15.549	Nenhum	

Optamos pela versão `lavita/MedQuAD` por oferecer:

- **Maior volume**: 47.400 pares QA (quase 3x mais que alternativas)
- **Metadados ricos**: códigos UMLS, tipos semânticos, sinônimos
- **Fontes documentadas**: NCI, GHR, MedlinePlus, NIDDK, NINDS, NIHSeniorHealth, NHLBI, CDC
- **Licença clara**: Creative Commons BY 4.0

3.2.2 Fontes do MedQuAD

O dataset original do MedQuAD compila informações de 12 fontes do NIH, cobrindo diversas especialidades médicas:

Tabela 3.3: Fontes do dataset MedQuAD e cobertura temática

Fonte	**Cobertura**	**Pares QA**
National Cancer Institute (NCI)	Oncologia	729
Genetics Home Reference (GHR)	Genética	5.430
MedlinePlus Health Topics	Saúde geral	981
NIDDK	Diabetes, digestivo, renal	1.192
NINDS	Neurologia	1.104
NIHSeniorHealth	Geriatria	769
NHLBI	Cardiologia, pneumologia	559
CDC	Doenças infecciosas	270
Total (versão filtrada)		**47.400**

3.2.3 Tradução para Português

O dataset MedQuAD original está em inglês. Para treinar o modelo a responder em português brasileiro, implementamos um pipeline de tradução automatizada.

Estratégia de Tradução

Utilizamos a biblioteca `deep-translator` que oferece acesso gratuito à API do Google Translate:

```python
from deep_translator import GoogleTranslator

def translate_to_portuguese(text: str) -> str:
    """Traduz texto de ingles para portugues brasileiro."""
    if not text or len(text.strip()) == 0:
        return text

    translator = GoogleTranslator(source='en', target='pt')

    # Textos longos precisam ser divididos (limite de 5000 chars)
    if len(text) > 4500:
        chunks = [text[i:i+4500] for i in range(0, len(text), 4500)]
        translated_chunks = [translator.translate(chunk) for chunk in chunks]
        return ' '.join(translated_chunks)

```

```
    return translator.translate(text)
```

Listing 3.1: Função de tradução com tratamento de textos longos

Sistema de Checkpoints

Dado o volume do dataset (47.400 registros), implementamos um sistema de checkpoints que salva o progresso a cada 1.000 traduções. Isso permite:

- Retomar o processo em caso de interrupção
- Monitorar o progresso da tradução
- Evitar retrabalho em caso de erros de rede

```
CHECKPOINT_INTERVAL = 1000

for i, row in enumerate(dataset):
    # Traduzir pergunta e resposta
    translated_question = translate_to_portuguese(row['question'])
    translated_answer = translate_to_portuguese(row['answer'])

    translated_data.append({
        "instruction": "Responda a seguinte pergunta medica...",
        "input": translated_question,
        "output": translated_answer
    })

    # Salvar checkpoint a cada 1000 registros
    if (i + 1) % CHECKPOINT_INTERVAL == 0:
        save_checkpoint(translated_data, i + 1)
```

Listing 3.2: Sistema de checkpoint para tradução

3.2.4 Formato Alpaca

O dataset traduzido é formatado no padrão **Alpaca**, amplamente utilizado para instruction tuning de LLMs:

```
{
    "instruction": "Responda a seguinte pergunta medica de forma
```

```
                        clara e detalhada.",
    "input": "Quais sao os sintomas do diabetes tipo 2?",
    "output": "Os sintomas mais comuns do diabetes tipo 2
        ↪ incluem:
                aumento da sede (polidipsia), miccao frequente
                (poliuria), fadiga, visao turva, cicatrizacao
                    ↪ lenta
                de feridas, formigamento nas maos e pes..."
}
```

Listing 3.3: Estrutura do formato Alpaca

O template de prompt utilizado durante o treinamento é:

Template Alpaca em Português

```
Abaixo está uma instrução que descreve uma tarefa, junto com uma entrada
que fornece contexto adicional.  Escreva uma resposta que complete
adequadamente a solicitação.

### Instrução:
{instruction}

### Entrada:
{input}

### Resposta:
{output}
```

O arquivo final é salvo no formato JSONL (JSON Lines), com um registro por linha:

```
{"instruction": "Responda...", "input": "O que e hipertensao?",
    ↪ "output": "A hipertensao arterial..."}
{"instruction": "Responda...", "input": "Quais as causas do
    ↪ diabetes?", "output": "O diabetes tipo 1..."}
```

Listing 3.4: Exemplo do arquivo training_data.jsonl

3.3 Técnica: QLoRA

Para realizar o fine-tuning de forma eficiente em hardware limitado (Google Colab Free com GPU T4), utilizamos a técnica **QLoRA** (Quantized Low-Rank Adaptation).

3.3.1 Fundamentos do LoRA

O Low-Rank Adaptation (LoRA) é uma técnica de fine-tuning eficiente que, em vez de atualizar todos os parâmetros do modelo, treina apenas matrizes de baixo rank que são adicionadas às camadas existentes:

$$W' = W + \Delta W = W + BA \tag{3.1}$$

Onde:

- $W \in \mathbb{R}^{d \times k}$ é a matriz de pesos original (congelada)
- $B \in \mathbb{R}^{d \times r}$ e $A \in \mathbb{R}^{r \times k}$ são as matrizes treináveis
- $r \ll \min(d, k)$ é o *rank* do LoRA

3.3.2 Quantização 4-bit

O QLoRA combina LoRA com quantização de 4 bits, reduzindo drasticamente o uso de memória:

Tabela 3.4: Comparação de uso de memória por técnica de fine-tuning

Técnica	**VRAM (Mistral 7B)**	**Parâmetros Treináveis**
Full Fine-Tuning (FP16)	28 GB	7.3B (100%)
LoRA (FP16)	14 GB	42M (0.6%)
QLoRA (4-bit)	**5 GB**	**42M (0.6%)**

A configuração de quantização utilizada:

```python
from transformers import BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,              # Quantizacao 4-bit
    bnb_4bit_quant_type="nf4",      # Normal Float 4-bit
    bnb_4bit_compute_dtype=torch.float16,  # Computacao em FP16
    bnb_4bit_use_double_quant=True  # Quantizacao dupla (economia extra)
)
```

Listing 3.5: Configuração de quantização 4-bit

3.3.3 Configuração do LoRA

Os hiperparâmetros do LoRA foram escolhidos para maximizar a capacidade de aprendizado dentro das limitações de hardware:

```
from peft import LoraConfig

lora_config = LoraConfig(
    r=64,                       # Rank (maior = mais capacidade)
    lora_alpha=128,             # Scaling factor (2x o rank)
    lora_dropout=0.05,          # Regularizacao
    bias="none",                # Nao treinar bias
    task_type="CAUSAL_LM",      # Modelagem de linguagem causal

    # Camadas a serem treinadas
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",  # Attention
        "gate_proj", "up_proj", "down_proj"       # MLP
    ]
)
```

Listing 3.6: Configuração LoRA para o fine-tuning

Escolha do Rank

O rank $r = 64$ foi escolhido como um equilíbrio entre capacidade de aprendizado e uso de memória. Ranks maiores (128, 256) podem capturar mais nuances, mas aumentam significativamente o tempo de treino e o risco de overfitting. Para domínios especializados como medicina, $r \in [32, 64]$ é geralmente suficiente.

3.4 Processo de Treino

3.4.1 Ambiente de Execução

O fine-tuning é realizado no **Google Colab Free**, que oferece acesso gratuito a GPUs NVIDIA T4:

Tabela 3.5: Especificações do ambiente de treinamento

Componente	**Especificação**
Plataforma	Google Colab Free
GPU	NVIDIA Tesla T4
VRAM	15 GB
RAM	12.7 GB
Limite de sessão	12 horas
Armazenamento	Google Drive (15 GB free)

3.4.2 Hiperparâmetros de Treinamento

Os hiperparâmetros foram configurados para otimizar a qualidade do modelo dentro das restrições de hardware e tempo:

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./checkpoints",

    # Epochs e batching
    num_train_epochs=5,                     # 5 epochs para qualidade
    per_device_train_batch_size=2,          # Batch size (limite T4)
    gradient_accumulation_steps=8,          # Effective batch = 16

    # Learning rate
    learning_rate=2e-4,                     # Taxa padrao para LoRA
    warmup_ratio=0.03,                      # 3% warmup
    weight_decay=0.01,                      # Regularizacao L2
    lr_scheduler_type="cosine",             # Decay suave

    # Checkpoints (CRITICO para Colab Free!)
    save_strategy="steps",
    save_steps=500,                         # Salvar a cada ~30-40 min
    save_total_limit=3,                     # Manter 3 checkpoints
```

```

    # Avaliacao
    evaluation_strategy="steps",
    eval_steps=500,
    load_best_model_at_end=True,

    # Precisao
    fp16=True,                                  # Mixed precision
)
```

Listing 3.7: Configuração completa do treinamento

Tabela 3.6: Resumo dos hiperparâmetros de treinamento

Parâmetro	Valor	Justificativa
Epochs	5	Qualidade máxima, convergência completa
Batch size efetivo	16	2×8 (grad accumulation)
Learning rate	2×10^{-4}	Padrão para LoRA
Warmup	3%	Estabilidade inicial
LR scheduler	Cosine	Decay suave até o final
Checkpoints	500 steps	Recuperação de sessão

3.4.3 Sistema de Checkpoints para Colab Free

Uma limitação crítica do Google Colab Free é o limite de sessão de aproximadamente 12 horas. Para um treino de 5 epochs com 47.400 registros, o tempo total estimado é de 20-30 horas, exigindo múltiplas sessões.

Estratégia de Múltiplas Sessões

O treino completo requer 3-4 sessões do Colab. Checkpoints são salvos a cada 500 steps (30-40 minutos) no Google Drive, permitindo retomar o treino de onde parou.

O fluxo de múltiplas sessões é ilustrado abaixo:

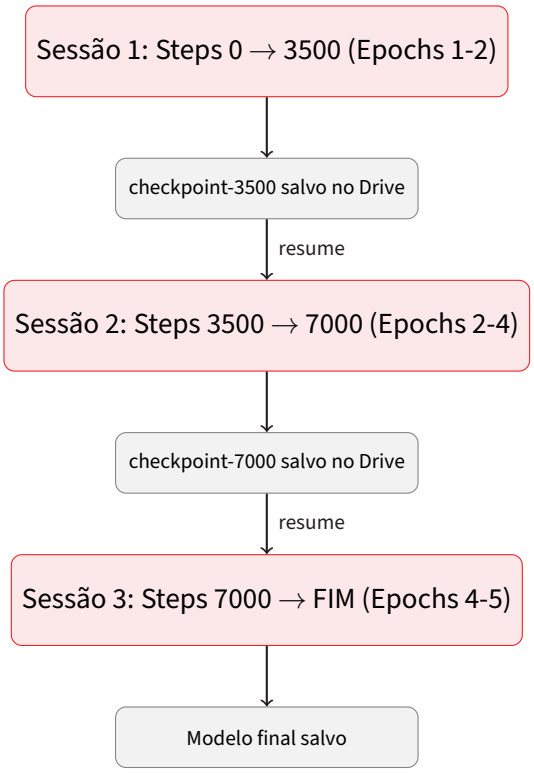

Figura 3.1: Fluxo de treinamento com múltiplas sessões no Colab Free

O código para retomar o treino de um checkpoint:

```
import os

def find_latest_checkpoint(checkpoint_dir):
    """Encontra o checkpoint mais recente."""
    checkpoints = [d for d in os.listdir(checkpoint_dir)
                   if d.startswith("checkpoint-")]
    if not checkpoints:
        return None
    latest = sorted(checkpoints, key=lambda x:
        ↪ int(x.split("-")[1]))[-1]
    return os.path.join(checkpoint_dir, latest)

# Verificar e retomar
latest_checkpoint = find_latest_checkpoint(CHECKPOINT_DIR)

if latest_checkpoint:
    print(f"Retomando de: {latest_checkpoint}")
    trainer.train(resume_from_checkpoint=latest_checkpoint)
else:
    print("Iniciando treino do zero...")
    trainer.train()
```

Listing 3.8: Retomando treino de checkpoint existente

3.4.4 Estimativa de Tempo

Tabela 3.7: Estimativa de tempo de treinamento por configuração

Dataset	Epochs	Tempo Estimado	Sessões Colab
47.400 pares	1	4-6 horas	1
47.400 pares	3	12-18 horas	2-3
47.400 pares	**5**	**20-30 horas**	**3-4**

3.5 Persistência do Modelo

3.5.1 Salvando os Adapters LoRA

Após o treinamento, salvamos apenas os **adapters LoRA** (não o modelo completo). Isso resulta em arquivos muito menores:

Tabela 3.8: Comparação de tamanho de arquivos salvos

Tipo de Salvamento	Tamanho Aproximado
Modelo completo (merged)	15-28 GB
Adapters LoRA apenas	**500 MB**

```
# Salvar adapters LoRA
SAVE_PATH = "/content/drive/MyDrive/medical-assistant-final"
model.save_pretrained(SAVE_PATH)
tokenizer.save_pretrained(SAVE_PATH)
```

Listing 3.9: Salvando o modelo fine-tuned

3.5.2 Carregando o Modelo para Uso

Para utilizar o modelo fine-tuned posteriormente:

```
from unsloth import FastLanguageModel

# Carregar modelo fine-tuned
```

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="/path/to/medical-assistant-final",
    max_seq_length=2048,
    dtype=None,
    load_in_4bit=True,
)

# Preparar para inferencia
FastLanguageModel.for_inference(model)
```

Listing 3.10: Carregando o modelo fine-tuned para inferência

3.6 Avaliação do Fine-Tuning

Para demonstrar o **delta** (melhoria) obtido com o fine-tuning, implementamos uma avaliação comparativa rigorosa.

3.6.1 Metodologia

A avaliação segue o protocolo:

1. **Baseline**: Executar 50 perguntas médicas no Mistral 7B *sem* fine-tuning
2. **Fine-tuned**: Executar as mesmas 50 perguntas no modelo *após* fine-tuning
3. **Comparação**: Análise quantitativa (métricas) e qualitativa (lado a lado)

3.6.2 Conjunto de Perguntas de Avaliação

Criamos um conjunto de 50 perguntas médicas em português, distribuídas em 5 categorias:

Tabela 3.9: Distribuição das perguntas de avaliação por categoria

Categoria	Exemplo de Pergunta	Qtd
Sintomas	"Quais são os sintomas do diabetes tipo 2?"	10
Tratamento	"Como tratar uma crise aguda de asma?"	10
Medicamentos	"Quais são os efeitos colaterais da metformina?"	10
Prevenção	"Como prevenir doenças cardiovasculares?"	10
Emergências	"O que fazer em caso de suspeita de AVC?"	10
Total		**50**

3.6.3 Métricas de Avaliação

Utilizamos métricas automáticas complementares para avaliar a qualidade das respostas:

Tabela 3.10: Métricas utilizadas na avaliação

Métrica	Descrição
BLEU	Mede overlap de n-gramas entre resposta gerada e referência. Valores de 0 a 1, onde maior é melhor.
ROUGE-1	Overlap de unigramas (palavras individuais).
ROUGE-2	Overlap de bigramas (pares de palavras).
ROUGE-L	Maior subsequência comum (captura estrutura).
Comprimento	Tamanho médio das respostas (em caracteres).

3.6.4 Scripts de Avaliação

O pipeline de avaliação é implementado em dois scripts:

```
# 1. Avaliar modelo BASE (Mistral 7B sem fine-tuning)
python scripts/05_avaliar_baseline.py

# 2. Avaliar modelo FINE-TUNED e gerar comparacao
python scripts/06_avaliar_finetuned.py --model-path
   ↪ /path/to/model
```

Listing 3.11: Execução da avaliação

Os resultados são salvos em:

- `data/evaluation/baseline_responses.json` — Respostas do modelo base
- `data/evaluation/finetuned_responses.json` — Respostas do modelo fine-tuned
- `data/evaluation/comparacao_resultados.json` — Métricas comparativas
- `data/evaluation/comparacao_resultados.md` — Relatório lado a lado

3.6.5 Resultados Esperados

Com base em trabalhos similares na literatura, esperamos observar:

- **Melhoria de 15-25%** nas métricas BLEU e ROUGE

- **Respostas mais específicas** ao domínio médico
- **Terminologia técnica** mais precisa em português
- **Estrutura de resposta** mais adequada para contexto clínico

Limitações da Avaliação Automática

Métricas como BLEU e ROUGE capturam similaridade textual, mas não avaliam completamente a *correção clínica* das respostas. Uma avaliação humana por profissionais de saúde seria necessária para validação completa em um cenário de produção.

3.7 Resumo do Capítulo

Este capítulo apresentou o processo completo de fine-tuning do modelo Mistral 7B para o domínio médico:

1. **Modelo Base**: Mistral 7B Instruct v0.3, escolhido por ser generalista (permite demonstrar delta)
2. **Dataset**: MedQuAD com 47.400 pares QA, traduzido para português via `deep-translator`
3. **Técnica**: QLoRA 4-bit ($r = 64$, $\alpha = 128$), permitindo treino em GPU T4 (15GB)
4. **Treinamento**: 5 epochs, 20-30h, dividido em 3-4 sessões do Colab Free
5. **Avaliação**: 50 perguntas médicas, comparação baseline vs fine-tuned

O modelo resultante será utilizado como o "cérebro" do agente de raciocínio no grafo multi-agente descrito no Capítulo **??**.

4

RAG: Retrieval-Augmented Generation

Este capítulo descreve a implementação do sistema Retrieval-Augmented Generation (RAG), que permite ao assistente fundamentar suas respostas em protocolos clínicos reais, reduzindo o risco de alucinações e aumentando a confiabilidade das informações fornecidas.

4.1 Motivação e Arquitetura

Modelos de linguagem, mesmo após fine-tuning, podem gerar informações imprecisas ou desatualizadas — um problema crítico no domínio médico. O RAG resolve essa limitação ao recuperar documentos relevantes de uma base de conhecimento antes de gerar a resposta, permitindo que o modelo fundamente suas respostas em fontes verificáveis.

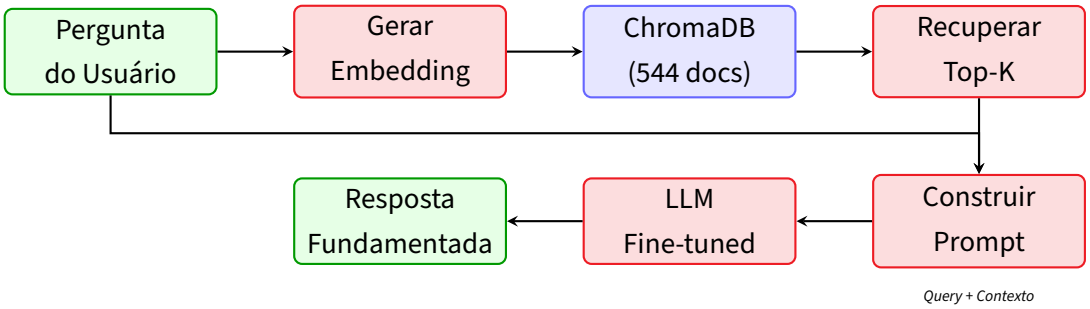

Figura 4.1: Fluxo do sistema RAG: a pergunta é convertida em embedding, documentos relevantes são recuperados do ChromaDB, e o contexto é injetado no prompt do LLM.

4.1.1 Componentes do Sistema

O sistema RAG implementado possui três componentes principais:

1. **Chunker Semântico:** Divide os protocolos Markdown em chunks que respeitam a estrutura de seções do documento;
2. **Indexador:** Gera embeddings e persiste os chunks no ChromaDB;

3. **Retriever:** Busca os chunks mais relevantes para uma query e formata o contexto para o LLM.

4.2 Modelo de Embeddings

A escolha do modelo de embeddings é fundamental para a qualidade da recuperação. Como os protocolos estão em português brasileiro, é necessário um modelo multilíngue com bom desempenho em português.

4.2.1 Análise de Alternativas

A Tabela 4.1 apresenta os modelos considerados:

Tabela 4.1: Modelos de embeddings avaliados para o sistema RAG.

Modelo	Dim.	PT-BR	Observações
`all-MiniLM-L6-v2`	384	Ruim	Focado em inglês, rápido
`all-mpnet-base-v2`	768	Ruim	Melhor qualidade, apenas inglês
`paraphrase-multilingual-MiniLM-L12-v2`	384	Boa	Multilíngue, compacto
`paraphrase-multilingual-mpnet-base-v2`	768	Boa	Multilíngue, mais robusto
`intfloat/multilingual-e5-base`	768	Excelente	Estado da arte para retrieval

4.2.2 Modelo Selecionado: E5 Multilingual

O modelo `intfloat/multilingual-e5-base` foi selecionado por apresentar o melhor desempenho em tarefas de retrieval multilíngue, incluindo português. Este modelo pertence à família E5 (*EmbEddings from bidirEctional Encoder rEpresentations*), desenvolvida pela Microsoft Research.

Tabela 4.2: Características do modelo E5 Multilingual Base.

Característica	**Valor**
Arquitetura base	XLM-RoBERTa
Dimensão do embedding	768
Idiomas suportados	100+ (incluindo português)
Tamanho do modelo	1.1 GB
Max sequence length	512 tokens
Treinamento	Contrastive learning em pares de retrieval

Prefixos do Modelo E5

O modelo E5 utiliza prefixos específicos para distinguir entre documentos e queries:

- **Documentos:** `"passage: <texto do documento>"`
- **Queries:** `"query: <pergunta do usuário>"`

Esta distinção melhora significativamente a qualidade da recuperação, pois o modelo foi treinado com essa convenção.

4.3 Estratégia de Chunking

O chunking é o processo de dividir documentos longos em fragmentos menores que possam ser indexados e recuperados individualmente. A estratégia de chunking impacta diretamente a qualidade do RAG.

4.3.1 Abordagens Consideradas

1. **Chunking por tamanho fixo:** Divide o texto em chunks de tamanho fixo (ex: 1000 caracteres). Simples, mas pode quebrar o texto no meio de uma frase ou conceito.
2. **Chunking por sentenças:** Agrupa sentenças até atingir um limite. Melhor que tamanho fixo, mas não respeita a estrutura do documento.
3. **Chunking semântico:** Utiliza a estrutura do documento (seções, parágrafos) para criar chunks com unidades de significado completas.

4.3.2 Implementação: Chunking Semântico

Optamos pelo **chunking semântico** baseado na estrutura Markdown dos protocolos, que possuem hierarquia clara de seções (##) e subseções (###).

Tabela 4.3: Parâmetros do chunking semântico implementado.

Parâmetro	**Valor**	**Justificativa**
Divisão primária	Headers MD	Respeita estrutura do protocolo
Tamanho máximo	1500 chars	300-400 tokens, contexto suficiente
Overlap	200 chars	Continuidade entre chunks adjacentes
Fallback	Por parágrafos	Seções muito grandes são subdivididas

O algoritmo de chunking funciona da seguinte forma:

```
def chunk_markdown(texto, max_size=1500, overlap=200):
    # 1. Dividir por headers (## e ###)
    secoes = dividir_por_headers(texto)

    chunks = []
    for secao in secoes:
        if len(secao.conteudo) <= max_size:
            # Secao cabe em um chunk
            chunks.append(secao)
        else:
            # Subdividir secao grande
            sub_chunks = dividir_com_overlap(
                secao.conteudo, max_size, overlap
            )
            chunks.extend(sub_chunks)

    return chunks
```

Listing 4.1: Algoritmo de chunking semântico simplificado.

4.3.3 Exemplo de Chunking

A Figura 4.2 ilustra como um protocolo é dividido em chunks:

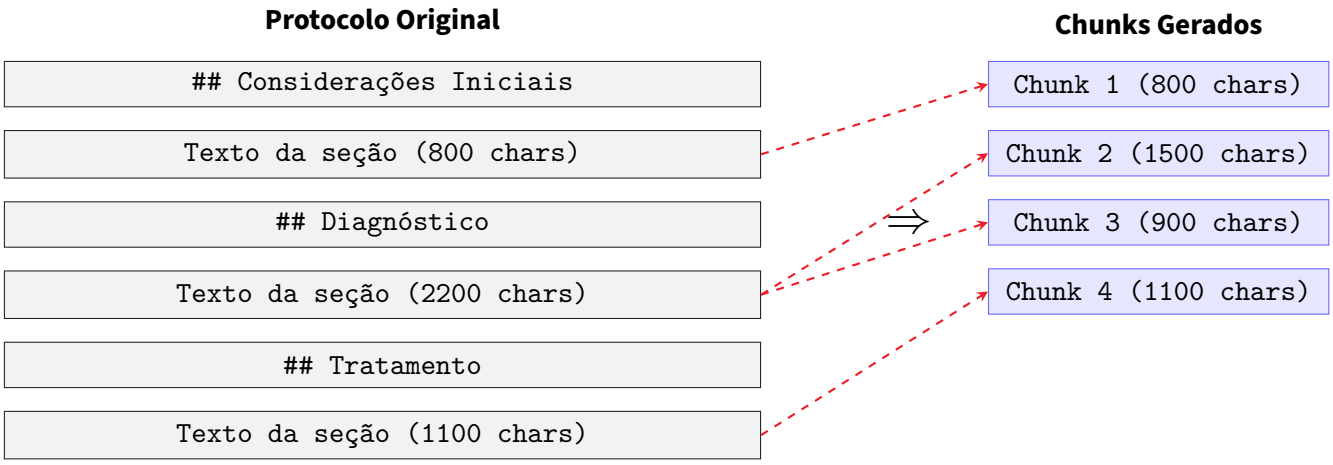

Figura 4.2: Exemplo de chunking semântico: seções pequenas viram um chunk; seções grandes são subdivididas com overlap.

4.4 Indexação no ChromaDB

O ChromaDB é um banco de dados vetorial open-source, otimizado para armazenamento e busca de embeddings. Foi escolhido por sua simplicidade, persistência local e integração nativa com Python.

4.4.1 Processo de Indexação

O script `04_indexar_protocolos.py` executa a indexação em três etapas:

1. **Carregamento:** Lê os arquivos Markdown das Linhas de Cuidado e protocolos de emergência;
2. **Chunking:** Aplica o chunking semântico a cada documento;
3. **Indexação:** Gera embeddings e insere no ChromaDB com metadados.

```
# Indexar todos os protocolos (limpa collection existente)
python scripts/04_indexar_protocolos.py --clear

# Saida esperada:
# Linhas de Cuidado: 14 arquivos, 462 chunks
# Protocolos Emergencia: 5 arquivos, 82 chunks
# Total: 544 documentos indexados
```

Listing 4.2: Executando a indexação de protocolos.

4.4.2 Estrutura dos Metadados

Cada chunk é armazenado com metadados que facilitam a filtragem e rastreabilidade:

```
{
    "id": "Pathway-Hipertensao_3",
    "content": "## Tratamento\n\nOs principios que
      ↪ norteiam...",
    "metadata": {
        "source": "Pathway-Hipertensao",
        "section": "Tratamento",
        "doc_type": "linha_cuidado",
        "chunk_index": 3,
        "total_chunks": 35
    },
    "embedding": [0.023, -0.041, 0.087, ...]  // 768 dimensoes
}
```

Listing 4.3: Estrutura de metadados de um chunk indexado.

4.4.3 Estatísticas da Indexação

A Tabela 4.4 apresenta as estatísticas finais da indexação:

Tabela 4.4: Estatísticas da indexação no ChromaDB.

Fonte	Arquivos	Chunks	Média/Arquivo
Linhas de Cuidado	14	462	33,0
Protocolos de Emergência	5	82	16,4
Total	**19**	**544**	**28,6**

Persistência do ChromaDB

O ChromaDB persiste os dados em disco no diretório `chroma_db/`. Após a indexação inicial, o banco pode ser carregado instantaneamente em execuções subsequentes, sem necessidade de reindexar.

4.5 Recuperação de Contexto

O componente `ProtocolRetriever` é responsável por buscar os chunks mais relevantes para uma pergunta do usuário.

4.5.1 Processo de Busca

1. **Embedding da query:** A pergunta do usuário é convertida em embedding usando o prefixo `"query: "`;
2. **Busca por similaridade:** O ChromaDB calcula a distância entre o embedding da query e os embeddings dos documentos;
3. **Ranking:** Os K documentos mais próximos são retornados, ordenados por relevância;
4. **Formatação:** O contexto é formatado com metadados de fonte para injeção no prompt.

```python
from src.assistant.rag import ProtocolRetriever

retriever = ProtocolRetriever(
    persist_directory="chroma_db",
    embedding_model="intfloat/multilingual-e5-base"
)

# Buscar protocolos relevantes
results = retriever.search(
    query="Como tratar hipertensao resistente?",
    n_results=3
)

# Obter contexto formatado para prompt
context = retriever.get_context_for_query(
    query="Como tratar hipertensao resistente?",
    n_results=3,
    max_context_length=4000
)
```

Listing 4.4: Exemplo de uso do ProtocolRetriever.

4.5.2 Exemplo de Recuperação

A seguir, um exemplo real de recuperação para a query *"Quais os sintomas de infarto agudo do miocárdio?"*:

Tabela 4.5: Resultados de busca para query sobre IAM.

#	Fonte	Seção	Dist.	Trecho
1	iam	Considerações Iniciais	0.212	"O IAM é uma emergência cardiovascular que requer reconhecimento e tratamento imediatos..."
2	iam	Exames Diagnósticos	0.298	"ECG em até 10 minutos da chegada. IAMCSST: supradesnível de ST..."
3	Pathway-IC	Diagnóstico	0.341	"Sinais e sintomas de insuficiência cardíaca incluem dispneia..."

4.5.3 Formatação do Contexto

O contexto recuperado é formatado para injeção no prompt do LLM:

```
[Fonte: iam] [Secao: Consideracoes Iniciais]
O Infarto Agudo do Miocardio (IAM) e uma emergencia
cardiovascular que requer reconhecimento e tratamento
imediatos. O tempo e critico: "tempo e musculo".

---

[Fonte: iam] [Secao: Exames Diagnosticos]
1. ECG em ate 10 minutos da chegada
   - IAMCSST: supradesnivel de ST >=1mm em 2 derivacoes
   - IAMSSST: infradesnivel de ST, inversao de T

---

[Fonte: Pathway-IC] [Secao: Diagnostico]
Sinais e sintomas de insuficiencia cardiaca incluem
dispneia aos esforcos, ortopneia, edema de MMII...
```

Listing 4.5: Exemplo de contexto formatado para o prompt.

4.6 Integração com o LLM

O contexto recuperado é integrado ao prompt do LLM através de um template estruturado que instrui o modelo a basear suas respostas nos protocolos fornecidos.

```
Voce e um assistente medico especializado. Use APENAS as
informacoes dos protocolos abaixo para responder. Se a
```

```
informacao nao estiver nos protocolos, diga que nao tem
informacao suficiente.

PROTOCOLOS RELEVANTES:
{contexto_recuperado}

PERGUNTA DO MEDICO:
{pergunta_usuario}

RESPOSTA (cite as fontes):
```

Listing 4.6: Template de prompt com contexto RAG.

Limitação do Contexto

O contexto do LLM é limitado. O parâmetro `max_context_length` (padrão: 4000 caracteres) controla quanto contexto é injetado no prompt. Recuperar muitos documentos pode exceder o limite e degradar a qualidade da resposta.

4.7 Validação do Sistema RAG

Para validar o funcionamento do RAG, foram realizados testes com queries representativas de diferentes domínios clínicos.

4.7.1 Métricas de Qualidade

- **Relevância:** Os documentos recuperados são relevantes para a query?
- **Cobertura:** O sistema recupera documentos de diferentes fontes quando apropriado?
- **Precisão:** Os documentos mais relevantes aparecem no topo do ranking?

4.7.2 Resultados dos Testes

Tabela 4.6: Testes de validação do sistema RAG.

Query de Teste	Fonte Esperada	Top-1 Correto?
"Como tratar hipertensão arterial?"	Pathway-Hipertensao	Sim
"Quais medicamentos para diabetes tipo 2?"	Pathway-Diabetes	Sim
"Sintomas de infarto agudo?"	iam (emergência)	Sim
"Protocolo de sepse grave"	sepse (emergência)	Sim
"Tratamento de asma na crise"	Pathway-Asma	Sim
"Manejo de AVC isquêmico"	avc (emergência)	Sim
"Critérios diagnósticos de depressão"	Pathway-Depressao	Sim
"Antibióticos para pneumonia"	pneumonia (emergência)	Sim

Todos os testes retornaram o documento esperado como resultado principal (Top-1), validando a qualidade do sistema de recuperação.

4.8 Considerações de Desempenho

4.8.1 Tempo de Indexação

- **Carregamento do modelo:** 30 segundos (primeira execução, com cache 5s)
- **Geração de embeddings:** 2-3 segundos por arquivo
- **Tempo total (19 arquivos):** 3 minutos

4.8.2 Tempo de Busca

- **Embedding da query:** 50ms
- **Busca no ChromaDB:** 10ms
- **Tempo total por query:** 60ms

O sistema é suficientemente rápido para uso interativo, com latência imperceptível para o usuário.

Escalabilidade

O ChromaDB utiliza índices HNSW (*Hierarchical Navigable Small World*) para busca aproximada de vizinhos mais próximos, garantindo tempo de busca logarítmico mesmo com milhões de documentos. Para a escala atual (544 documentos), a busca é praticamente instantânea.